Università di Catania

DIPARTIMENTO DI INGEGNERIA
ELETTRICA ELETTRONICA E INFORMATICA

CORSO DI LAUREA MAGISTRALE IN INGEGNERIA INFORMATICA

CloudFG

Studenti:
Ficarra Filippo 1000034832
Guerrera Mario Anthony 1000035663

Anno Accademico 2025/2026

Sommario

1. Descrizione progetto 1
2. Modulo Client C# 2
3. Modulo Server Go 4
4. Modulo Analitics Python 6
5. Flussi di esecuzione 7
6. Avvio Applicazione 8

1. Descrizione progetto

L'applicazione CloudFG simula un servizio di storage cloud, permettendo l'archiviazione e la gestione di file, organizzati per titoli scelti dall'utente e autore (cioè l'utente loggato che carica i file) ed indicizzati dall' hashcode del file. Inoltre, permette l'analisi di documenti testuali (solo con estensioni .txt, .docx e .pdf), al fine di ottenere report analitici dettagliati, basati su indici di leggibilità e statistiche lessicali.

Le funzionalità del progetto sono:

- Autenticazione utente (login/registrazione).
- Caricamento documenti, con inserimento di un titolo scelto dall'utente.
- Ricerca e Visualizzazione documenti caricati.
- Download/Apertura locale.
- Eliminazione file e Account.
- Analisi testuale automatica dei documenti supportati (txt, pdf e docx).

L'architettura è composta dai seguenti moduli:

- Modulo Client in C#.
- Modulo Server in Go.
- Modulo di analisi testuale automatica in Python.

2. Modulo Client C#

Il modulo funge da interfaccia utente, grazie all'utilizzo del framework WPF (Windows Presentation Foundation), incluso in .NET. Esso, grazie al linguaggio dichiarativo XAML, permette di definire l'interfaccia della nostra applicazione.

Il modulo è formato dalle seguenti finestre:

- App.xaml/ App.xaml.cs: rappresenta il punto di accesso del modulo. Contiene la logica di sessione.
- LoginWindow.xaml/ LoginWindow.xaml.cs: permette la registrazione o autenticazione dell'utente.
- MainWindow.xaml/ MainWindow.xaml.cs: costituisce il core dell'app. Permette la ricerca e il caricamento dei file, oltre alla gestione dell'account.
- UploadDetailsWindow.xaml/ UploadDetailsWindow.xaml.cs: permette l'inserimento di dettagli relativi al file al momento del caricamento, come il titolo che si vuole attribuire al file da caricare.
- SearchWindow.xaml/ SearchWindow.xaml.cs: è la finestra di presentazione dei risultati di ricerca.
- Analitics.cs e Document.cs: definiscono i dati dei file e delle sue metriche. Servono per mappare i nomi dei campi JSON con i campi delle classi C#.

All'avvio del client, il programma:

- Legge il token salvato in locale, nel formato <username-timestamp>.
- Considera valida la sessione se il timestamp assegnato in fase di login è minore di 300 secondi (5 minuti).
- Apre la MainWindow, se valida, altrimenti rimanda alla LoginWindow e pulisce il token.

Nella LoginWindow vi sono due *TabItem*:

- *Login*: invia username e password al server GO all'indirizzo *http://localhost:8080/login*
- *Registrati*: invia username e password al server GO all'indirizzo *http://localhost:8080/register*

In caso di successo, viene restituito il token e viene aperta la MainWindow. In caso di errore, viene mostrato un messaggio con il motivo dell'errore (credenziali non valide, username già in uso, server non raggiungibile).

L'utente, dalla main window, può selezionare un file e inviarlo al server all'indirizzo http://localhost:8080/upload, tramite richiesta POST, passando come parametri il file, il titolo scelto dall'utente per quel file, e l'autore (che sarà lo username dell'utente). Durante l'upload, l' interfaccia utente disabilita il pulsante di caricamento dei file e mostra una progress bar, insieme ad un messaggio sullo stato del caricamento (successo/errore/duplicato).

Dalla stessa finestra è possibile ricercare un documento tramite il titolo o visualizzare tutti i documenti dell'utente. Entrambe aprono una SearchWindow che caricherà i risultati (attraverso Data Biding) richiesti al server tramite:

- http://localhost:8080/search_all?user={username}
- http://localhost:8080/search?query={query}&user={username}

I due vengono usati, rispettivamente, per visualizzare tutti i file di quel determinato utente e per effettuare una query sui titoli (sempre relativi a quell'utente).

Per ogni documento sono mostrati: titolo, autore, path del file, data e ora di upload. Le azioni disponibili sono:

- Apri: richiede il file al server tramite richiesta all'indirizzo http://localhost:8080/download?hash={libro.Hash}&user={username}, salva il file in una directory temporanea e apre il file con un programma predefinito attraverso l' oggetto ProcessStartInfo.
- Elimina: chiede conferma della cancellazione, manda la richiesta di eliminazione del file al server all' indirizzo http://localhost:8080/delete?hash={libro.Hash}&user={username} e aggiorna la lista risultati. Ovviamente, il file viene eliminato sia dal database, sia dal file system del server.
- Visualizza Analisi: richiede le statistiche sul documento, se disponibili, all'indirizzo http://localhost:8080/download_analitics?hash={libro.Hash} e le mostra attraverso un popup. Le statistiche salvate sono:
    - Indice di Gulpease.
    - Numero di caratteri.
    - Numero di parole.
    - Numero di frasi.
    - Tempo di lettura stimato.
    - Tempo di elaborazione delle statistiche.
    - Numero di parole uniche (ricchezza lessicale).

Infine, dalla MainWindow, è possibile effettuare il logout tramite il pulsante "Esci", che cancellerà il token e mostrerà la LoginWindow, ed eliminare l'account tramite richiesta all'indirizzo http://localhost:8080/delete_user?user={username}, che cancellerà il token e mostrerà la LoginWindow. Ovviamente, quando un account viene eliminato, tutti i file di quell'utente vengono rimossi dal database e dal file system del server.

3. Modulo Server Go

Il modulo server si occupa di memorizzare i documenti (nel file system e nel database), e gestirli tramite SQLite, con un database *CloudFG.db* contenente tre tabelle:

- *'users'*: memorizza username e password (hashata) degli utenti registrati.
- *'files'*: memorizza hash, autore, titolo, timestamp di upload, dimensione e file path di salvataggio per ogni singolo file memorizzato sul server.
  La tabella files ha come chiave primaria la coppia (hash, autore) in quanto è permesso che un utente possa caricare lo stesso identico file che altri utenti hanno già memorizzato sul server.
- *'analitics'*: memorizza metriche per ogni file salvato. La tabella Analitics utilizza l'hash del file come chiave primaria, in modo che, anche se più copie dello stesso file si trovano sul server, le metriche rimangono uguali.

Per interfacciarsi con il db, e permettere quindi le operazioni CRUD sulle tabelle, espone diverse API.

3.1 API di Autenticazione:

- POST /register:
  - valida il JSON passato nel body della Request.
  - verifica che i campi username e password non siano vuoti.
  - se lo username non è già in uso, salva username e l'hash della password nella tabella *users*.
  - restituisce un token nel formato <username, timestamp> che verrà memorizzato in locale dal client.
- POST /login:
  - valida il JSON passato nel body della Request.
  - verifica che l' hash della password immessa corrisponda all'hash della password corrispondente a quel determinato username.
  - se corretto restituisce il token, altrimenti un messaggio di errore.

3.2 API di upload dei file:

- POST /upload:
  - valida il JSON passato nel body della Request, legge il *file* e i suoi metadati.
  - se nel file system del server è già presente un file con lo stesso nome, allora esso viene salvato con un ulteriore valore *(i),* dove *i* è il numero di file con lo stesso nome presenti sul file system. Infatti, è permesso l'inserimento di file con lo stesso nome (per esempio, l'inserimento di un file modificato), purchè l'hash sia diverso (se l'hash è uguale, vuol dire che pure il contenuto è uguale).
  - apre il file, legge il suo contenuto a blocchi di 32KB per non saturare le risorse del server e memorizza il file nel file system locale del server.
  - durante la lettura viene contemporaneamente calcolato l'hash del file.
  - impedisce duplicati del file per uno stesso autore.

    - registra i metadati del file e le altre informazioni nella tabella *'files'*.
    - avvia una goroutine, avviando il modulo python per le statistiche.

3.2 API di ricerca ed elenco dei file:

- GET /search?query={query}&user={username}
    - restituisce le informazioni sui file dell'utente *username* con *title* LIKE query.
- GET /search_all?user={username}
    - restituisce tutti le informazioni sui file dell' utente *username*.

3.3 API di download:

- GET /download?hash={libro.Hash}&user={username}
    - recupera il pathname del file dal database.
    - invia il file al client.

3.4 API di eliminazione:

- DELETE /delete?hash={libro.Hash}&user={username}
    - seleziona dalla tabella *'files'* il pathname del file con hash e autore corrispondenti alla richiesta.
    - elimina il record dal database.
    - rimuove il file dal file system locale del server.
    - se nessun altro utente ha salvato sul server lo stesso file elimina anche le analitics corrispondenti all'hash del file.
- DELETE /delete_user?user={username}
    - elimina tutti i file e i relativi dati dell'utente dalla tabella *'files'* e dal filesystem.
    - controlla, per ogni file, se è presente un altro utente che ha salvato lo stesso identico file e in caso di esito negativo elimina il record dalla tabella *'analitics'*.
    - elimina l'utente dal db.

3.5 API sulle analitics

- POST /upload_analitics:
    - riceve un JSON, dal modulo python, con hash file e le metriche
    - salva le metriche nella tabella *'analitics'*.
- GET /download_analitics?hash={hash}
    - restituisce le metriche, associate all'hash richiesto, al client.

4. Modulo Analitics Python

Il modulo Analitics, scritto in python, entra in gioco nel momento in cui il server avvia la Goroutine e manda in esecuzione, sulla stessa macchina, il file main.py.

Il flusso di esecuzione del modulo Analitics è strutturato in:

- Ricezione, dal server, del pathname del file appena salvato.
- Calcolo dell'hash del file.
- Estrazione del testo dai file supportati.
- Calcolo delle seguenti metriche linguistiche sul file:
    - numero di parole.
    - Caratteri.
    - tempo di lettura stimato.
    - indice di Gulpease (specifico per l'italiano).
    - numero di parole uniche (ricchezza lessicale).
- Calcolo, attraverso un decorator, del tempo necessario all'analisi del testo.
- Invio dei risultati al server Go, all'indirizzo http://localhost:8080/upload_analitics

I formati supportati per l'analisi sono pdf, docx e txt.

Se il formato non è supportato, non vengono prodotte le analisi sul testo, ma è comunque possibile caricare il file sul server, visualizzarli e aprirli nel client.

5. Flussi di esecuzione

5.1 Flusso di registrazione:

1. L'utente, attraverso la LoginWindow, immette username e password. L'app costruisce un oggetto C# anonimo con i campi Username e Password.
2. L'oggetto viene convertito in JSON e inviato a POST /register.
3. Il server Go decodifica il body della request nel tipo AuthRequest(Username, Password) e salva le credenziali sul database.
4. Il server risponde con il codice *201:Created* e un token nel formato <username, timestamp>.
5. Il client salva il token in Properties.Settings e apre la MainWindow.

5.2 Flusso di login:

1. Il client invia lo stesso payload (vedasi flusso di registrazione) JSON a POST /login.
2. Il server confronta la password hashata con quella salvata nella tabella *users*.
3. Se valida restituisce il codice *200:OK* con il token.
4. Se non valida restituisce *401:Unauthorized* ed il messaggio di errore.

5.3 Flusso di upload:

1. L'utente seleziona il file attraverso la MainWindow e inserisce il titolo attraverso la UploadDetailsWindow.
2. Il client invia al server un form con il file, il titolo e l'autore a POST /upload.
3. Il server salva il file in locale e memorizza i metadati nel database.
4. Il server avvia la goroutine eseguendo il modulo python in background e risponde con l'esito del caricamento.

5.4 Flusso di analisi python:

1. Il server go avvia Analitics/main.py passando come argomento il pathname del file.
2. Python:
    1. Calcola l'hash 256 del file.
    2. Estrae il testo.
    3. Calcola le metriche.
3. Python invia le metriche a POST /upload_analitics con payload JSON annidato Data{hash, Analitics{...}}.
4. Il server Go decodifica il JSON e salva le metriche sul database.

5.5 Flusso di ricerca/apertura:

1. Il client richiede l'elenco dei propri file (filtrato o non) inviando a GET /search?query={query}&user={username}.
2. Il server esegue la query SQL con *title LIKE '%query%' AND author='user'*.
3. Restituisce un array JSON di documenti.
4. Il client può:
    - aprire il file in locale tramite GET /download?hash={libro.Hash}&user={username}.

- recuperare le analitics sul documento, se disponibili, tramite /GET /download_analitics?hash={hash}. Il server leggerà dalla tabella *analitics* e risponderà con un JSON al client che mostrerà i valori in un MessageBox.

5.6 Flusso di eliminazione file:

1. Il client invia la richiesta DELETE /delete?hash={libro.Hash}&user={username}.
2. Il server chiama la funzione Delete(hash, author) che:
    1. Elimina il record in *files*.
    2. Elimina il file fisico dal server.
    3. Verifica se esistono altri file con lo stesso hash. Se non esistono elimina anche le analitics relative al file.
3. Il client ricarica la ricerca per aggiornare i risultati.

Flusso di eliminazione account:

1. Il client invia la richiesta DELETE /delete_user?user={username}.
2. Il server recupera tutti gli hash dei file caricati dall'utente.
3. Per ogni hash chiama la funzione Delete(hash, author).
4. Il server elimina l'utente dalla tabella *users*.
5. Il client pulisce il token e torna a LoginWindow.

6. Avvio Applicazione

Per avviare il sistema sono possibili due modalità, specificate all'interno del README.md:

1. Avvio automatico tramite Start.ps1
2. Manuale:
    1. Si avvia il server Go
    2. Si configura l'ambiente python e le dipendenze
    3. Si avvia il Client C#